

Teoría de Algoritmos

(75.29) Curso Buchwald - Genender

Trabajo Práctico 2:

Programación Dinámica For The Win

---

9 de octubre de 2026

Lucio Romero Mascaró
113091

Valentín Raunic
114081

Índice

1. Resumen **3**

2. El Problema **4**

2.1. Introducción 4

2.2. Planteamiento 4

2.3. Consecuencias de la optimalidad 5

3. Teoría Programación Dinámica **6**

3.1. Programación Dinámica aplicada al problema 7

3.2. Pensamiento y utilidades 9

4. Diseño del Algoritmo **10**

4.1. Lógica 10

4.2. Código 10

5. Analizando el Algoritmo **14**

5.1. Complejidad 17

5.2. Demostración de la Optimalidad 17

5.2.1. Impacto de la Variabilidad en la Optimalidad 19

5.3. Experimentos 19

6. Conclusión **21**

Bibliografía **21**

1. Resumen

Se presenta un resumen del Trabajo Práctico 2, detallando los temas a tratar para ofrecer una lectura fluida, clara y precisa que permita comprender el desarrollo en su totalidad.

En primer lugar, se va a hablar sobre el problema propuesto, cómo fue el planteo de la resolución y cómo se logró llegar a que ésta misma pueda ser óptima para cualquier caso posible.

Se detalla la teoría necesaria sobre los *algoritmos de Programación Dinámica*, enfocándose en lo más básico de la teoría, entre ella la *Ecuación de Recurrencia*, la forma de ver los subproblemas, la toma de decisiones y la técnica de memorización. Para ver estos aspectos aplicados, se incluye un breve ejemplo de la idea en general y su analogía a conceptos en el mundo real (**como el QUE SE YO**).

Por otro lado, se detalla el diseño del algoritmo implementado en *pseudocódigo*, explicando anteriormente la lógica, cómo fue buscada y cómo fue pensada esa resolución. Adjuntándose al final la implementación realizada.

Se realiza un *análisis* del algoritmo planteado, mediante el estudio de la complejidad y una demostración matemática para validar la optimalidad del algoritmo, de forma clara y con ejemplos propuestos. Se incorporan múltiples experimentos empíricos de forma tal que se pueda visualizar gráficamente las medidas de tiempo sobre cierta cantidad de casos.

Finalmente, se expone una conclusión sobre lo aprendido, la dificultad del problema, y la particularidad de estos algoritmos y su pensamiento.

2. El Problema

Mateo tiene 7 años. Los mismos años que tenía Sophia cuando comenzaron a jugar al juego de las monedas. Eso quiere decir que Mateo también ya aprendió sobre algoritmos greedy, y lo comenzó a aplicar. Esto hace que ahora quién gane dependa más de quién comience y un tanto de suerte.

Esto no le gusta nada a Sophia. Ella quiere estar segura de ganar siempre. Lo bueno es que ella comenzó a aprender sobre programación dinámica. Ahora va a aplicar esta nueva técnica para asegurarse ganar siempre que pueda.

El juego de las monedas consiste en disponer una fila de n monedas, de diferentes valores. En cada turno, un jugador debe elegir alguna moneda. Pero no puede elegir cualquiera: *sólo puede elegir o bien la primera de la fila, o bien la última.* Al elegirla, la remueve de la fila, y le toca luego al otro jugador, quien debe elegir otra moneda siguiendo la misma regla. Siguen agarrando monedas hasta que no quede ninguna. Quien gane será quien tenga el mayor valor acumulado (por sumatoria).

Para transformar esta situación en un problema algorítmico concreto, el desafío es el siguiente: Hacer un análisis del problema, y proponer un algoritmo de Programación Dinámica que obtenga la solución óptima al problema planteado: Dada la secuencia de monedas m_1, m_2, ..., m_n, sabiendo que Mateo siempre elegirá la moneda más grande para sí en su respectivo turno, indicar qué monedas debe ir eligiendo Sophia para si misma, de tal forma que Sophia gane siempre que pueda, y que siempre alcance el *mayor valor acumulado posible*. Considerar que Sophia siempre comienza.

2.1. Introducción

Tranquilamente, se puede observar que los objetivos son dos: que *Sophia gane siempre que pueda* y que *alcance el mayor valor acumulado posible*. A eso sumémosle que vamos a saber que Mateo siempre va a elegir la moneda de mayor valor en su turno.

Por último, lo más importante es que la elección va a ser si o sí entre 2 monedas y el valor entre ellas. Se puede preguntar:

- ¿Cómo se podría hacer que eligiendo x moneda (una entre esas dos), no haya otra moneda próxima (mayor a x) que Mateo la elija en el turno siguiente?
- ¿Cómo es posible crear un algoritmo de Programación Dinámica en el cual sea óptimo para cualquier caso de monedas? ¿Qué es un algoritmo de Programación Dinámica y qué significa que sea óptimo?
- ¿Qué casos van a ser que Sophia pierda? ¿Es posible que Sophia gane siempre?

Una vez propuesto eso, sigamos pensando para ahora empezar con el planteo.

2.2. Planteamiento

Hacer que gane siempre que pueda Sophia, y tenemos la suerte también de que Sophia tiene *visión* por sobre todas las monedas y *no tiene el sesgo* de elegir siempre la mayor. Entonces, cómo

podemos aconsejar a Sophia de tal forma que si siempre hace lo que le hayamos dicho, gane siempre que pueda y se quede con el mejor valor acumulado posible?

La primer solución que se tiene que ocurrir es que entre las dos monedas a elegir agarre siempre la mayor en valor. Eso va a garantizar que siempre termine con una suma mayor que Mateo. O no?

Pero las monedas que elige Mateo, ¿van a llegar a repercutir mucho en el problema?

Para que Sophia gane, tiene que tener la mayor cantidad de valor acumulado comparado con Mateo, entonces si Sophia agarra por cada par siempre la mayor, Sophia va a competir con las monedas de mayor valor que agarre Mateo. La manera *avariciosa* de Sophia, copiándole a Mateo, la va a llevar por mal camino.

Simple contraejemplo: la fila de monedas son [5, 10, 1, 2]. Turno Sophia, entre [5, 2] agarra la de mayor valor [5]. Turno Mateo, entre [10, 2] agarra la de mayor valor tambien [10]. Ahora sin importar la que agarre Sophia, va a terminar *perdiendo y sin el mayor valor acumulado.*

La manera avariciosa, manera *Greedy*, pierde la visión del "Big Picture" del problema. Ahora planteo, ¿Qué pasaría si Sophia empeora su estado actual, mejorando su óptimo global?. Algo que no ocurriría nunca con el pensamiento greedy, puede utilizarse a nuestro favor con un pensamiento cercano a la **Programación Dinámica**. Volviendo al ejemplo anterior, primer turno Sophia agarra la moneda de valor [2], en vez de la mayor, "empeorando" su estado actual. Mateo, en su turno y de manera inocente, agarra la de mayor valor entre [5, 1]. En el turno de Sophia va a elegir entre [10, 1]. Eligiendo la [10] y asegurándose la victoria + el mayor valor acumulado posible.

¿Cómo se puede aplicar esa técnica, para cualquier fila de monedas?

¿Existe un patrón que se cumpla siempre?

2.3. Consecuencias de la optimalidad

Si gana siempre que pueda y siempre obtenga el mayor valor acumulado posible, significa que:

- Las indicaciones propuestas fueron óptimas.
- Si pasamos las instrucciones a un algoritmo, ese mismo es óptimo.
- Si es posible demostrar un solo contraejemplo con respecto al mayor valor acumulado, significa que no era óptimo[1].
- Va a haber que demostrar matemáticamente que es óptimo y que no gana siempre.

Las pruebas a realizar de todo esto y mucho más se va a realizar en la lectura que sigue. Se recomienda leer detenidamente cada paso, y si no se entiende, releerlo.

A eso sumarle el pensamiento tipo Programación Dinámica, hasta acá se dijo sutilmente como es éste. A continuación, se explica exáctamente eso: qué es un **algoritmo de Programación Dinámica**, qué técnica realiza, y más que nada cómo se puede implementar a este problema propuesto.

[1] La condición de que siempre obtenga el mayor valor acumulado, es obligatoria. No tiene por qué ganar siempre, ya hablaremos de esto en el futuro.

3. Teoría Programación Dinámica

"El pasado puede doler, pero tal como yo lo veo, puedes huir de él o aprender de él". - Rafiki (El Rey León, 1994).

Programación Dinámica es una técnica de programación (como los algoritmos Greedy). Ayuda a dejar de repetir cálculos aprovechando los ya hechos, usar resultados pasados para resolver el futuro. La solución por Programación Dinámica brinda resultados, estos resultados calculados los podemos guardar, lo que nos puede ayudar a optimizar el tiempo de ejecución. Esa técnica de guardado se llama *Memoization* o *Memorización*.

La Programación Dinámica explora implícitamente el espacio de posibles soluciones, lo cual siempre va a convenir ver el problema general en pequeños *subproblemas* del mismo. Nos basamos en la intuición que nos da la Memoization para reconocer los subproblemas y utilizarlos para construir la solución. Una vez que tenemos todas las soluciones memorizadas, el problema está resuelto. Nos ayuda evitar explorar un espacio exponencial de soluciones por fuerza bruta, y de esa manera podemos reducir la complejidad temporal del algoritmo a implementar.

Los *subproblemas* establecen una lógica inductiva:

- "Si yo puedo asumir que ya tengo calculados todas las soluciones anteriores (casos más pequeños), ¿Cómo puedo usarlos para construir la que necesito?"

Esto nos lleva a pensar en un cierto *patrón* que pueden generar los subproblemas y, en consecuencia, una *Ecuación de Recurrencia*[2] que cumpla con el armado de los subproblemas.

¿Qué pasaría si, buscando un valor anterior para encontrar la solución requerida, viene la persona más importante y que más sabe de este tema a decirte: "Esta es la solución que necesitas."? Confiás plenamente, pero ¿cómo utilizarías esta información a tu favor?

Ejemplo:

Secuencia de Fibonacci. Para un número n, no es necesario calcular todas las ramas de sus anteriores, porque se van a *repetir* resultados. Viene esta persona y te dice: "Para los 2 casos que quieras sumar, ya calculé los resultados y son estos". Entonces, simplemente hay que sumar esos dos *óptimos* que nos brindó, que sabemos que son correctos y no dudamos en ningún momento. ¿Cuál es la forma de los subproblemas? ¿Cuál es la ecuación de recurrencia?

$$\mathcal{OPT}(n) = \mathcal{OPT}(n-1) + \mathcal{OPT}(n-2)$$

Siendo $OPT(n)$ el resultado de fibonacci que queremos calcular.

Si $n = 5$, calcula $fibonacci(5) = fibonacci(4) + fibonacci(3)$

Entonces, si *iterativamente* tenemos precalculados, recordados, *memorizados* todos los resultados que queremos averiguar, los fibonacci anteriores a n, en una simple suma obtenemos el resultado requerido.

Para la resolución de los problemas utilizando la técnica de Programación Dinámica se prefiere mucho la forma iterativa (bottom-up), y no utilizar la forma recursiva (top-down). Similar a los algoritmos Greedy. Esto es debido a que:

[2] Una ecuación de recurrencia en el ámbito de la informática es una expresión matemática que define el tiempo de ejecución o el costo de un algoritmo en función de sí mismo para entradas más pequeñas.

1. Es mucho más fácil de entender qué esta pasando, cuándo y cómo.
2. Como consecuencia de lo anterior, se facilita el cálculo de la complejidad (temporal y espacial).
3. La resolución de problemas con esta técnica suelen ser más fáciles de resolver de forma inductiva que recursiva.
4. Nos estructura mucho más cómo pensar la solución[3].
5. Se pueden aplicar optimizaciones de memoria fácilmente.

¿Como se va a saber que puedo utilizar Programación Dinámica en un problema?

- Hay un número polinomial de subproblemas
- La solución al problema original puede ser construido a partir de soluciones a subproblemas
- Hay un orden natural de los subproblemas de menor a mayor. Los subproblemas "mayores" son resueltos mediante la composición de problemas "menores".

Otra utilidad que nos sirve usar la técnica de *Memoization*,comúnmente, permite pasar de un tiempo de ejecución exponencial a uno polinomial. Además, nos permite en ciertos ejercicios una optimización de memoria: En el anterior ejemplo de fibonacci, se puede utilizar un arreglo con todos los óptimos de 0 hasta n, para recordar las soluciones. Pero mejor aún, utilizar una optimización y guardar solamente 2 variables recordando las últimas dos soluciones. Eso nos va a brindar una complejidad espacial constante de $O(1)$.

Otra sección relacionada a la técnica de Programación Dinámica, es la *reconstrucción del problema*. Si yo quiero recordar y devolver todos los resultados del fibonacci en el orden en que los usé, ahí sí se utiliza un arreglo con todos los óptimos precalculados y en una recorrida se pueden obtener qué resultados y *cuándo* los utilicé. La reconstrucción del problema es construir la solución de abajo hacia arriba.

3.1. Programación Dinámica aplicada al problema

En el contexto del juego de las monedas, el desafío de Sophia es planificar la mejor estrategia. El futuro es planificable y predecible, ya que sabemos que Mateo agarra siempre la moneda de mayor valor que se le cruce en frente.

Entonces, la decisión de Sophia en el turno actual no reduce el tamaño del problema en 1 moneda, sino en 2 monedas (cada ronda). Su propia elección sumada a la reacción avariciosa de Mateo.

El Pensamiento de Sophia:
En vez de elegir sin pensar la moneda de mayor valor (como haría un algoritmo Greedy), Sophia piensa mentalmente los dos universos posibles en su turno:

1. **¿Qué pasa si elijo la moneda de la izquierda?** Mateo, por instinto greedy, va a agarrar la mayor de las dos monedas que queden en los extremos. Sabiendo exactamente cuál va a elegir Mateo, Sophia se pregunta: *¿Cuántos puntos máximos me aseguro con las monedas que sobran en la mesa?.*

[3] Primero la ecuación de recurrencia, después la programación del problema (siendo este último paso, sencillo)

2. **¿Qué pasa si elijo la moneda de la derecha?** Mateo va a hacer exactamente lo mismo. Sophia vuelve a calcular: *En este otro caso, ¿cuántos puntos me aseguro para el futuro?*.

Sophia simplemente calcula el total de cada camino (la moneda que elige ahora + los puntos que ya sabe que va a ganar en el futuro) y ejecuta la opción que le devuelva el número más grande.

Ecuación de Recurrencia:
Para formalizar este pensamiento, definimos a $\mathcal{OPT}(\text{prim}, \text{ult})$ como el máximo valor acumulado que Sophia puede asegurarse en la fila de monedas actual.

Casos Base (y comúnmente, el final del juego):

- Si queda sólo una moneda: Sophia se la queda. $\mathcal{OPT}(\text{prim}, \text{prim}) = m[\text{prim}]$
- Si quedan dos monedas: Sophia elige la mayor. $\mathcal{OPT}(\text{prim}, \text{ult}) = \text{máx}(m[\text{prim}], m[\text{ult}])$

Caso General (Cuando quedan 3 o más monedas):
Lo mejor es simplemente el máximo entre esos dos futuros casos que calcula Sophia mentalmente:

$$\mathcal{OPT}(\text{prim}, \text{ult}) = \text{máx} \begin{cases} m[\text{prim}] + \text{Futuro si agarré la Primera} \\ m[\text{ult}] + \text{Futuro si agarré la Última} \end{cases}$$

Matemáticamente, ese "futuro"depende del Greedy aplicado de Mateo, quedando la ecuación completa así:

$$\mathcal{OPT}(\text{prim}, \text{ult}) = \text{máx} \begin{cases} m[\text{prim}] + \begin{cases} \mathcal{OPT}(\text{prim}+2, \text{ult}) & \text{si Mateo elige } m[\text{prim}+1] \\ \mathcal{OPT}(\text{prim}+1, \text{ult}-1) & \text{si Mateo elige } m[\text{ult}] \end{cases} \\ m[\text{ult}] + \begin{cases} \mathcal{OPT}(\text{prim}+1, \text{ult}-1) & \text{si Mateo elige } m[\text{prim}] \\ \mathcal{OPT}(\text{prim}, \text{ult}-2) & \text{si Mateo elige } m[\text{ult}-1] \end{cases} \end{cases}$$

Demostración de mayor valor acumulado posible:

Para demostrar que esta estrategia es la mejor que le podemos recomendar a Sophia y garantiza siempre el mayor valor acumulado posible para Sophia, nos basamos en el razonamiento inductivo.

Como vimos en los Casos Base, es un obvio que Sophia hace la decisión perfecta cuando quedan 1 o 2 monedas en la fila. El éxito de la estrategia de Programación Dinámica se basa en que, cuando Sophia se enfrenta a un problema de 3, 4 o n monedas, ella no improvisa. Al evaluar el çamino izquierdoz el çamino derecho", la ecuación consulta los resultados de los subproblemas más chicos (el futuro). Ventaja superior comparado con el pensamiento Greedy de agarrar siempre la de mayor valor.

Como esos subproblemas del futuro fueron resueltos previamente construidos desde los casos base hacia arriba, iterativamente, Sophia está sumando su moneda actual con un futuro que es matemáticamente insuperable. Dado que la ecuación evalúa el 100 % de las ramas posibles y siempre extrae la función máx() entre ellas, es lógicamente imposible que exista una combinación de elecciones en la fila que sume un valor mayor al que la estrategia da.

Se va a verificar, por demostración matemática y con ejemplos, en una sección a continuación.

3.2. Pensamiento y utilidades

El Pensamiento de Programación Dinámica es análisis. La mente comúnmente tiende a abrumarse con el futuro, esta mentalidad introduce: entender que los problemas gigantes no son bloques fijos, sino un árbol de subproblemas vinculados por el tiempo. Adoptar esta forma de razonar implica aprender a pensar de atrás hacia adelante. En lugar de pararse en el casillero inicial y avanzar a ciegas tanteando opciones a ver que onda, el "pensador dinámico" se para conceptualmente en la meta final que quiera y reconstruye el camino de forma inversa. Y lo mejor de todo, las personas hacen esto todo el tiempo en el día a día cuando planifican su vida, partiendo sus objetivos grandes en pequeñas acciones concretas hasta llegar a la primera decisión que tiene al alcance para hacer hoy. (Aristóteles hablando sobre la deliberación en Ética nicomáquea[4])

Esta forma de razonar retroespectiva tiene una base analítica muy fuerte: si el futuro está planificado de manera óptima, el presente se resuelve solo. Trasladado a las decisiones de la vida real, esto nos da una estructura clara para manejar la optimización del esfuerzo del día a día y la satisfacción aplazada a través de "ecuaciones de recurrencia personales". El que adopta este chip mental entiende que las decisiones actuales son variables que modifican el escenario del mañana. Por eso, se acepta de forma natural un costo, un esfuerzo extra o un rendimiento subóptimo en el plano inmediato (como aguantarse de salidas por estudiar o meterle horas a un proyecto) con la certeza matemática de que es el único camino válido para maximizar el resultado global a largo plazo. (Principio de Optimalidad de Bellman)

En un plano cognitivo, el enfoque PD introduce la *memoization* como una técnica estricta para guardar la experiencia personal. El pensamiento dinámico no deja el desperdicio del aprendizaje ni el desgaste de tropezar dos veces con la misma piedra. Si o si exige que cada crisis superada, cada error corregido y cada subproblema resuelto quede anotado en una suerte de "tabla mental" permanente. Al cruzarse con situaciones repetitivas en la rutina diaria, la mente no reinventa la lógica desde cero, sino que hace una consulta rápida a su memoria. Y así, la toma de decisiones cotidianas no permiten el azar y pasa a pasa a ser algo más razonable, donde lo vivido se transforma de forma directa en un mapa predictivo para resolver la incertidumbre.

[4]Expone en el Libro III de esta obra que la deliberación no empieza por el principio, sino por la meta. Analizamos los requisitos de fin a principio, descubriendo que "el último paso en la resolución es el primero en la ejecución".

4. Diseño del Algoritmo

. Logica explicar logica y como llegamos a eso explicar por qué el código fue hecho de esa manera

Escribite un par de for y estamo

¿Por qué se elige para Sophia la mayor de las monedas y para Mateo la menor entre cada par? Si la regla hubiera omitido la maximización de la distancia, una posible regla greedy podría ser: "Para cada par, se elige la mayor", pero con eso mismo se le está dando la opción a Mateo de tener mayor puntaje. Y además el riesgo de Sophia, de que eligiendo la moneda de mayor valor en el momento, puede seguir después de esa una mayor a la elegida.

A veces la intuición falla, por eso hubo que pensar bien la técnica a realizar. Habiendo ya hablado de la secuencia de elecciones que se realizan para llegar a la solución del problema (Sophia gana), hay que pasar esa parte a la *lógica algorítmica*. ¿Cuál es el paso a paso en cualquier caso de monedas? ¿Cuál puede ser el peor caso posible? ¿Cómo interactúa el algoritmo con los datos del problema? ¿Cómo se pasan los datos del problema al idioma computacional?

4.1. Lógica

Una fila de n monedas, se puede implementar con un simple arreglo de números enteros. Donde cada posición representa una moneda, con su valor asociado. El turno va a empezar siendo de Sophia, y es donde va a empezar la elección.

La lógica se gestiona utilizando dos punteros que delimitan los extremos disponibles de la fila: `inicio` (inicializado en la primera posición) y `fin` (inicializado en la última). En cada turno, se evalúan los valores apuntados por los dos extremos. Después de cada elección, el puntero correspondiente se desplaza hacia el centro de la fila, reduciendo el espacio de búsqueda (porque ya seleccionamos dicha moneda).

Una vez que terminan las selecciones de una ronda, el control se alterna de forma automática. Siguiendo el marco teórico *greedy*, esta secuencia iterativa sigue hasta que el juego termina, escenario que ocurre cuando los punteros se cruzan, indicando que ya no quedan monedas disponibles en la mesa para seleccionar. Cada valor de la moneda seleccionada fue guardada en una variable `puntos` para ir guardando los resultados obtenidos.

4.2. Código

mostrar el pseudocódigo le puedo poner colorcitos? o muy de bot

```
def juego(monedas [1 , 2 , 3 ,... , n ]):
    // Si no hay monedas, ambos tienen 0 puntos
    if len(monedas) == 0:
        Devuelvo (0, 0)

    // Cantidad de Monedas
    n = len(monedas)

    // Armo la matriz de n x n
    OPT = [[0] * n for _ in range(n)]
```

```
11
12      // Casos base donde elijo entre una o dos monedas (reescribilo vos esto porque
        me quedo mal)
13      for i in range(n):
14          OPT[i][i] = monedas[i] // Asigno para la posicion (i, i) a la moneda i
15          if i < n -1:
16              // Si el rango me permite, asigno al optimo de la posicion (i, i+1) al
        mas grande entre la moneda actual y la que sigue
17              OPT[i][i+1] = max(monedas[i], monedas[i+1])
18
19      // Caso General
20      for long in range(3, n+1):
21          // 'long' es la longitud del problema o distancia entre i y j, lo hago
        crecer desde una longitud [3, n] (n incluido)
22          for i in range(n - long + 1):
23              j = i + long -1
24
25              // Caso 1: Sophia plantea elegir la primer moneda -> el M[i]
26              if monedas[i+1] > monedas[j]:
27                  // Sophia agarra la moneda i. Mateo agarra la moneda de mayor valor
         entre i+1 y j
28                  ganancia_izq = monedas[i] + OPT[i+2][j]
29              else:
30                  ganancia_izq = monedas[i] + OPT[i+1][j-1]
31
32              // Caso 2: Sophia plantea elegir la ultima moneda -> M[j]
33              if monedas[i] > monedas[j-1]:
34                  // Sophia agarra la moneda j. Mateo agarra la moneda de mayor valor
         entre i y j-1
35                  ganancia_der = monedas[j] + OPT[i+1][j-1]
36              else:
37                  ganancia_der = monedas[j] + OPT[i][j-2]
38
39              // Ecuacion de recurrencia
40              OPT[i][j] = max(ganancia_izq, ganancia_der)
41      // Armo la guia o manual de pasos que debera seguir Sophia para poder sacar la
        mayor cantidad de puntos
42      guia = _reconstruccion(monedas, OPT)
43      return partida(monedas, guia)
44
45  def reconstruccion(monedas [1 , 2 , 3 ,... , n ], OPT [Matriz de optimalidad n x n
        ]):
46      // Que paso funciona como una especie de guia o manual que tiene Sophia, el
        cual le indica si agarrar la primera o la ultima
47      QUE_PASO = []
48      primera = 0
49      ultima = n -1
50
51      turno_sophia = True
52
53      while primera <= ultima:
54          // Mientras haya monedas para ver
55          if turno_sophia:
56              if primera == ultima:
57                  // Si es el turno de Sophia y solo queda una, es indistinto cual
        agarrar
```

```
                QUE_PASO.append(0)
                break

            // Me fijo si Sophia hubiera elegido la primer moneda
            if monedas[primera+1] > monedas[ultima]:
                ganancia_izq = monedas[primera] + (OPT[primera+2][ultima] if (primera+2) <= j else 0) // Siempre que pueda elijo el optimo, sino 0
            else:
                ganancia_izq = monedas[primera] + (OPT[primera+1][ultima-1] if (primera+1) <= j-1 else 0)

            // Me fijo si coincide gracias a la ecuacion de recurrencia al reves
            if OPT[primera][ultima] == ganancia_izq:
                QUE_PASO.append(0)
                primera += 1
            else:
                QUE_PASO.append(1)
                ultima -= 1
        else:
            // Turno de mateo, al pensar de manera Greedy, elije la mas grande disponible siempre que pueda.
            if monedas[primera] > monedas[ultima]:
                primera += 1
            else:
                ultima -= 1

        // Cambio de turno
        turno_sophia = not turno_sophia

    Devuelvo QUE_PASO // No hace falta invertirlo. Esta en orden crlonlogico


def partida(monedas: list[int], QUE_PASO: list[int]):
    puntos_mateo = 0
    puntos_sophia = 0
    primera = 0
    ultima = n - 1
    turno_sophia = True
    // Paso a seguir del manual
    paso = 0
    while primera <= ultima:
        if turno_sophia: // Cuando es el turno de Sophia, uso el manual o guia para saber que tengo que hacer
            if QUE_PASO[paso] == 1: // Si es 1, significa que debo tomar la moneda de la derecha
                moneda = monedas[ultima]
                ultima -= 1
            else:
                moneda = monedas[primera] // Si no (o sea es 0), debo tomar la moneda de la izquierda
                primera += 1
            puntos_sophia += moneda
            paso += 1
        else:
            if monedas[primera] > monedas[ultima]: // Cuando es el turno de Mateo, simplemente elije la moneda de mayor valor
```

```
107                 moneda = monedas[primera]
108                 primera += 1
109             else:
110                 moneda = monedas[ultima]
111                 ultima -= 1
112             puntos_mateo += moneda
113         turno_sophia = not turno_sophia
114 
115     Devuelvo (puntos_sophia, puntos_mateo)
```

5. Analizando el Algoritmo

. Complejidad temporal cómo se llegó a eso y las ventajas de PD espacial explicar la estrcutura utilizada y el por qué. . Demostración de la Optimalidad usar la ec. de recurrencia para demostrar que llega al maximo valor acumulado posible demostracion por induccion ez no me tengo que olvidar de poner ejemplos . Impacto de la Variabilidad en la Optimalidad puede Mateo ganar mas veces? . Experimentos mediciones de tiempo ver cómo afecta a los tiempos del algoritmo la variabilidad de los valores de las monedas hacer mediciones mas exhaustivas (y no solo el ajuste por cuadrados minimos)

Para analizar el algoritmo propuesto anteriormente, va a resultar útil plantear un ejemplo para analizar el comportamiento: Sea el arreglo de monedas $A = [96{,}594, 437, 674, 950]$. Creamos la tabla o matriz para poder ir anotando los optimos, ademas la llenamos de ceros.

indices	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	0	0	0	0
3	0	0	0	0	0
4	0	0	0	0	0

- Una vez esta la matriz creada, vamos recorriendo cada moneda, la moneda 0 ira en la posicion (0, 0), la moneda 1 en la posición (1, 1) y asi sucesivamente.
- Además, una vez tengamos la moneda i, en todos los pasos (excepto el final), habremos de comparar la moneda i con su siguiente, o sea i+1, para asi ubicar la mayor de estas en la posicion (i+1)

indices	0	1	2	3	4
0	96	594	0	0	0
1	0	594	594	0	0
2	0	0	437	674	0
3	0	0	0	674	950
4	0	0	0	0	950

- En este caso podemos observar como para i = 0, en la posición (0, 1) ubicamos al 594, ya que el mismo es mas grande que su anterior, el 96. Este mismo paso con todos las monedas (menos la ultima).
- Una vez tenemos la primer parte hecha, se prosigue a la segunda, donde hay que aplicar la ecuación de recurrencia. Al ser un algoritmo del tipo bottomUp, vamos a ir resolviendo el problema grande arrancando por los problemas mas chicos, esos empiezan utilizando unicamente tres monedas.
- Habiendo dicho eso, iniciamos con los subproblemas. Tenemos tres monedas y las mismas son [96, 594, 437], ademas tenemos que i = 0 y j = 2. Primero evaluamos la ganancia que podriamos llegar a tener si elegimos la moneda de la izquierda, esta se compone de elegir el 96 y saber que mateo eligirá la mas grande entre [594, 437], por lo que nos quedaria como ganancia izquierda = 96 + 437, o lo que es lo mismo, 96 + OPT[0+2][2].

- Por otro lado, la ganancia derecha sera el resultado de tomar la ultima moneda, de valor 437, por lo que mateo se que eligira la mayor entre [594, 96] quedando nuevamente 437 + 96, o 437 + OPT[0][2-2]. Siendo que entonces el maximio aca es igual en ambos casos.

indices	0	1	2	3	4
0	96	594	533	0	0
1	0	594	594	0	0
2	0	0	437	674	0
3	0	0	0	674	950
4	0	0	0	0	950

- La distancia entre i y j sigue siendo de 3, pero ahora i = 1 y j = 3. Con esos datos analizamos la lista que contiene a [594, 437, 674], inicialmente elegimos a 597, y mateo eligira 674, por lo que hacemos 594 + OPT[1+1][3-1] (o sea 437), quedando una ganancia izquierda de 1031.
- Yendo hacia el final ahora, elegimos 674, Mateo eligirá 594 por ser mas grande que 437, entonces volvere a sumar 674 + OPT[1+1][2-1] (437), quedando una ganancia derecha de 1111 puntos. Siendo que entonces entre estos dos, Sophia eligira los 1111 puntos en la posicion OPT[1][3].

indices	0	1	2	3	4
0	96	594	533	0	0
1	0	594	594	1111	0
2	0	0	437	674	0
3	0	0	0	674	950
4	0	0	0	0	950

- Ahora i = 2 y j = 4, con la siguiente lista como material [437, 674, 950]. Tomando el elemento de la izquierda tenemos que, como mateo tomara el 950 por sobre el 674, nos queda 437 + OPT[2+1][4-1] (674), siendo igual a 1111.
- Por el otro lado, si tomamos la ultima, o sea la moneda de valor 950, nos queda que, como Mateo tomara la de 674, sumaremos 950 + OPT[2][4-2] (437) siendo igual a 1387. Tomando como maximo entre la ganancia por izquierda y por derecha, a la ganancia por derecha, guardando ese valor en OPT[2][4]

indices	0	1	2	3	4
0	96	594	533	0	0
1	0	594	594	1111	0
2	0	0	437	674	1387
3	0	0	0	674	950
4	0	0	0	0	950

- Ahora si, el tamaño del subproblema crece, i pasa a estar a 3 numeros de j, por lo cual arrancamos con i = 0 y j = 3. Teniendo para trabajar la lista de [96, 594, 437, 674]. Si elegimos el 96, sabremos que mateo eligira entre el 594 y el 674, prevaleciendo en su decision el tomar el 674, quedandonos 96 + OPT[0+1][3-1] (594) dando como ganancia izquierda 690 puntos.

- Del otro lado, tomando 674, Mateo eligira 437 por sobre 96, por lo que nos queda como ganancia derecha la cuenta de 674 + OPT[0][3-2] (594) siendo esto igual a 1268 puntos. Y como ademas es mayor al resultado anterior, nos quedaremos con esta opcion, alojandola en OPT[0][3].

indices	0	1	2	3	4
0	96	594	533	1268	0
1	0	594	594	1111	0
2	0	0	437	674	1387
3	0	0	0	674	950
4	0	0	0	0	950

- En este caso, i = 1 y j = 4, siendo la ultima iteracion de este caso, donde ahora tenemos la lista [594, 437, 674, 950].
- Arrancando con el lado izquierdo, tomamos el 594 y lo sumaremos al OPT[1+1][4-1], ya que sabemos que Mateo se quedara con la ultima moneda. Esto nos da una ganancia izquierda de 1268.
- Para el otro caso, tomamos el 950 y nos fijamos que Mateo se quedara con el 674 por sobre el 594, entonces haremos 950 + OPT[1][4-2] (594), quedando como ganancia derecha 1544, que al ser mayor a la ganancia izquierda, la guardaremos en OPT[1][4].

indices	0	1	2	3	4
0	96	594	533	1268	0
1	0	594	594	1111	1544
2	0	0	437	674	1387
3	0	0	0	674	950
4	0	0	0	0	950

- Finalmente, en el ultimo paso, i = 0 y j = 4, acá tenemos toda la lista a disposición, por lo que comenzamos por calcular la ganancia izquierda, tomando al 96 y sumando OPT[0+1][4-1] (1111), esto ya que Mateo eligira 950 por sobre el 594, teniedno una ganancia izquierda de 1207 puntos.
- Tomo el 950 y como el maximo entre 96 y 674 es 674, sumo OPT[0][2] (533) y termina quedando como ganancia derecha 1483 puntos, siendo superior a la ganancia izquierda, por lo que para terminar de llenar la tabla, en OPT[0][4] pondremos 1483.

indices	0	1	2	3	4
0	96	594	533	1268	1483
1	0	594	594	1111	1544
2	0	0	437	674	1387
3	0	0	0	674	950
4	0	0	0	0	950

Se puede analizar el algoritmo implementado de varias formas, que vamos a estar explicando a continuación. Entre ellas, se va a hablar de la complejidad temporal y espacial algorítmica del algoritmo, y si es posible mejorarla.

También se habla de la optimalidad del algoritmo, anteriormente se hizo una breve presentación de ésta la cual ahora se va a explayar más y se va a demostrar matemáticamente por el método inductivo.

Por último se van a realizar experimentos sobre el mismo algoritmo, poniendolo sobre arreglos de distintos tamaños para hacer y visualizar mediciones de tiempo distintos.

5.1. Complejidad

Cuando se habla de *complejidad algorítmica*, buscamos en el algoritmo analizar el algoritmo de dos maneras: *espacial* y *temporal*. La complejidad espacial determina el espacio adicional ocupado en memoria por las estructuras de datos auxiliares, mientras que la complejidad temporal evalúa el tiempo de ejecución del algoritmo a medida que el tamaño de instancia (cantidad de monedas) crece. ¿Cómo se comporta el algoritmo?

Para este análisis se adopta el enfoque del *peor caso posible*[5], el cual se formaliza matemáticamente mediante la notación Big-Oh ($\mathcal{O}$).

La **complejidad temporal** del algoritmo diseñado para representar la sesión de juego de Sophia y Mateo es de $\mathcal{O}(n^2)$, donde n es la cantidad inicial de monedas en el arreglo. Esto es asi, ya que hay dos bucles o for anidados, uno externo que recorre las longitudes de los subproblemas desde 3 hasta n, y por cada bucle externo se ejecutan los internos, que recorre n - long + 1 posiciones. Si bien esto podria parecer como que es mucho, en realidad es un algoritmo muy eficiente por lo que hace, gracias al concepto de **Memoization**, el cual consiste en que en vez de ir calculando una y otra vez subproblemas que ya resolviste previamente, acá no es necesario, ya que se guarda el resultado de cada subproblema, por lo que si necesitamos volver a saber un valor, simplemente es cuestion de ir a buscarlo sabiendo su ubicacion, o sea una complejidad de $\mathcal{O}(1)$. Si no usaramos la tecnica de **Memoization** estariamos hablando de una complejidad de $\mathcal{O}(2^n)$, ya que deberiamos recalcular constantemente valores que ya calculamos previamente. Pero usar esta tenica tiene un defecto, el espacio utilizado.

Faltaria agregar el recuento"de complejidades, lo hago mañana

Por otro lado, el análisis de la **complejidad espacial** del algoritmo implementado, se puede considerar un poco más directo. Al usar la tecnica de memoization y ademas ser un problema que tiene dos variables (primera, ultima), termina requiriendo de un espacio que permita guardar los resultados de los distintos subproblemas, requiriendo en este caso una matriz de $n \times n$, quedando una complejidad espacial de $\mathcal{O}(n^2)$

5.2. Demostración de la Optimalidad

Que el algoritmo planteado sea *óptimo* implica que la estrategia *greedy* es suficiente para garantizar que Sophia gane siempre matemáticamente (obtener una sumatoria de valores mayor o igual a la de Mateo) en cualquier escenario posible, independientemente de la distribución de los valores. Se va a sacar de esta demostración el caso borde de un arreglo de tamaño n par donde todos los elementos son iguales, ya que va a resultar en un empate.

[5] Es importante destacar que en este algoritmo greedy, la distribución o el orden de los valores de las monedas no altera el analisis. Por lo tanto, el peor caso, el mejor caso y el caso promedio son iguales en términos numéricos de operaciones ejecutadas, lo cual simplifica la validación experimental.

Cuando se habla de algoritmos greedy, su demostración de que sean óptimos puede ser un tanto difícil. Una vez demostrada y habiendo justificado que sea óptimo, se puede asumir que el mismo algoritmo lleva una de las mejores soluciones de "manera" greedy.

Para demostrar que funcione siempre la regla greedy propuesta, se va utilizar la *Demostración por Inducción Matemática*. Se define nuestra variable de inducción k como la cantidad de *rondas* del juego. En cada ronda, se extraen exactamente 2 monedas (la elección de Sophia y la elección de Sophia sobre Mateo). Por lo tanto, un juego con $2k$ monedas equivale a k rondas.

Hipótesis Inductiva $P(k)$: Dada una fila de $2k$ monedas, la estrategia greedy garantiza que la ganancia acumulada de Sophia (S_{total}) será mayor a la ganancia acumulada de Mateo (M_{total}). Es decir, $S_{total} > M_{total}$ (gracias a haber descartado el empate).

Paso Base ($k = 1$):
El juego contiene $2(1) = 2$ monedas, guardadas en el arreglo $A = [c_1, c_2]$ (c de coin). Sophia, de manera avariciosa bajo su regla greedy, elegirá el máximo de los extremos. Su ganancia será $S_1 = \text{máx}(c_1, c_2)$. En consecuencia, el arreglo se reduce a una sola moneda, la cual Mateo está obligado a tomar. Su ganancia será $M_1 = \text{mín}(c_1, c_2)$. Al descartar el caso del empate (que esas mismas 2 monedas sean de igual valor), se cumple que:

$$\text{máx}(c_1, c_2) > \text{mín}(c_1, c_2) \implies S_1 > M_1$$

El caso base $P(1)$ es verdadero.

Paso Inductivo: ($P(k+1)$)
Se asume que $P(k)$ es verdadero. Eso significa que, para cualquier arreglo de $2k$ monedas, la estrategia garantiza que $S_{k_rondas} > M_{k_rondas}$. Ahora se tiene que demostrar que $P(k+1)$ también es verdadero para un arreglo inicial de $2(k+1) = 2k+2$ monedas.

En la primera ronda del juego con $2k+2$ monedas, los extremos disponibles son c_1 y c_{2k+2}. Sophia aplica la regla y elige $S_{ronda_1} = \text{máx}(c_1, c_{2k+2})$. Supongamos que elige c_1. El arreglo se reduce a los elementos desde c_2 hasta c_{2k+2}.

Después, Sophia ejecuta el turno de Mateo aplicando la regla greedy de maximizar la distancia para él: elegir el mínimo de los nuevos extremos disponibles (c_2 y c_{2k+2}). Mateo gana $M_{ronda_1} = \text{mín}(c_2, c_{2k+2})$.

Por la definición de la función mínimo[6], sabemos que $M_{ronda_1} \leq c_{2k+2}$. A su vez, como Sophia eligió c_1 por ser el máximo inicial, sabemos que $c_1 \geq c_{2k+2}$. Se puede deducir que:

$$S_{ronda_1} = c_1 \geq c_{2k+2} \geq \text{mín}(c_2, c_{2k+2}) = M_{ronda_1}$$

$$S_{ronda_1} \geq M_{ronda_1}$$

Luego de esta primera ronda, quedan exactamente $2k$ monedas en la mesa. Por nuestra **Hipótesis Inductiva**, se sabe que si ambos jugadores continúan aplicando la misma estrategia sobre estas $2k$ monedas restantes, se cumplirá que la ganancia de las rondas que siguen será $S_{resto} \geq M_{resto}$.

[6] La función mínimo, en el ámbito de matemáticas y programación, simplemente toma un grupo de números y devuelve el más pequeño de todos.

La ganancia total del juego será la suma de la primera ronda más el resto de las rondas:

$$S_{total} = S_{ronda_1} + S_{resto}$$

$$M_{total} = M_{ronda_1} + M_{resto}$$

Y como se ha demostrado que $S_{ronda_1} \geq M_{ronda_1}$, que no existe la opción de empate y asumido que $S_{resto} \geq M_{resto}$, al sumar ambas desigualdades se concluye que:

$$S_{total} > M_{total}$$

Por lo tanto, si $P(k)$ es verdadero, $P(k+1)$ también lo es. Queda demostrado por inducción que Sophia siempre va a obtener un puntaje mayor al de Mateo. En caso de que el tamaño inicial del arreglo n sea impar, después de $\lfloor N/2 \rfloor$ rondas completas, Sophia tendrá el turno final para tomar la moneda restante $c_{final} \geq 0$, lo que incrementará aún más su puntaje final, manteniendo la desigualdad $S_{total} > M_{total}$.

5.2.1. Impacto de la Variabilidad en la Optimalidad

Siguiendo el análisis del algoritmo, lo vamos a poner frente a una variabilidad grande entre los valores de las monedas. Por ejemplo, una distribución exponencial como $A = [1, 10, 100, 1000]$ con una varianza asimétrica[7], es indispensable destacar que la optimalidad no se ve afectada en ningún momento. La demostración por inducción desarrollada en la sección anterior no depende de la magnitud de los valores, sino de su relación de orden (mayor, menor o igual).

Dado que la regla greedy propuesta evalúa las desigualdades locales en cada turno (donde siempre se cumple que $c_1 \geq \text{mín}(c_2, c_{2k+2})$), y recordar que la estrategia greedy *no ve* las decisiones del impacto de valores futuros. Sin importar la diferencia de valor entre dos monedas es de valor 1 o de 1,000,000, Sophia siempre asegura matemáticamente la cota superior en la elección de esa ronda. Por lo tanto, la variabilidad del conjunto de datos *afecta la escala del puntaje final*, pero es imposible de invertir la condición de victoria $S_{total} > M_{total}$.[1, 3, 4, 9, 6, 2]

5.3. Experimentos

aa **MOSTRAR** aaa MOSTRARQUESI NO USAMOS MEMOIZATION SE NOS VA AL PINGOOOOO

Se realizaron distintos experimentos en función de crear veinte arreglos de distintos tamaños, cada uno con una cantidad de elementos que van desde cien hasta diez millones de elementos equiespaciados, por lo que la separación entre cada número es de aproximadamente 526.310 números. Luego cada arreglo va a recibir números pseudoaleatorios que van desde el cero hasta cien mil (módulo `random`).

Como se observa en la Figura 1, la medida presenta una superposición prácticamente exacta con la recta de ajuste de Cuadrados Mínimos[8]. Esto evidencia un comportamiento lineal respecto

[7] La varianza es una medida estadística que indica qué tan dispersos están los datos de un conjunto respecto a su valor promedio. El concepto de varianza asimétrica (o semivarianza) se utiliza cuando solo importa medir el riesgo o la dispersión en una dirección específica.

[8] La técnica de Cuadrados Mínimos es un método de optimización matemática que, en este análisis, permite

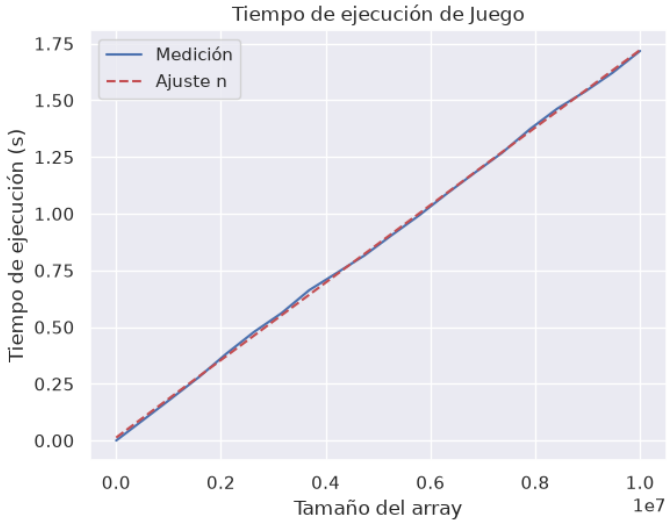


Figura 1: Tiempo de ejecución en función de la cantidad de elementos (n).

al incremento de elementos en el arreglo, confirmando que la complejidad temporal es $\mathcal{O}(n)$.



Figura 2: Error absoluto del ajuste por Cuadrados Mínimos.

En la Figura 2 se refuerza el argumento, observando que el error absoluto se mantiene acotado y en magnitudes despreciables a lo largo de todo el rango de n, validando la complejidad temporal del algoritmo *greedy* implementado.

encontrar la función lineal que mejor se ajusta al conjunto de datos experimentales, *minimizando* la suma de los errores al cuadrado entre los *tiempos de ejecución* medidos y la recta $\mathcal{O}(n)$.

6. Conclusión

El desarrollo de este trabajo práctico permitió analizar las capacidades y limitaciones del paradigma *Greedy* aplicado al problema. La implementación de una regla greedy asimétrica, asimétrica porque Sophia maximiza su beneficio local inmediato y minimiza el de Mateo. Demostró ser una solución óptima, logrando una complejidad temporal lineal $\mathcal{O}(n)$ y espacial constante $\mathcal{O}(1)$ que fue corroborada usando el ajuste de Cuadrados Mínimos.

Igualmente, el análisis más relevante de este escenario se basa en comprender las condiciones estrictas que habilitan la optimalidad del enfoque greedy. La estrategia implementada garantiza la victoria absoluta de Sophia únicamente por una restricción crucial de la consigna: ella tiene el control total sobre las decisiones de Mateo, anulando su capacidad de jugar a favor de él.

En conclusión, el problema propuesto los fundamentos teóricos en el diseño de algoritmos: las soluciones greedy son herramientas "poderosas", elegantes y rápidas, pero su éxito está vinculado a la estructura del entorno. Una pequeña modificación en las reglas del juego o en el comportamiento de los actores puede llegar a transformar un algoritmo lineal óptimo en una estrategia deficiente por el problema de no tener la visión del "Big Picture".

Se pudo demostrar que la estrategia greedy es un gran diseño y una gran solución, pero también un pensamiento peligroso al ignorar las fronteras del óptimo global.

Bibliografía

- Bellman, nosedespuesmefijo (1978). *Principio de Optimalidad ¿?* Quilmes: Lucio soy un crack.
- Buchwald, Martín y Genender Ezequiel (2026). *Teoría de Algoritmos (75.29) - Apuntes de cátedra y clases*. Facultad de Ingeniería, Universidad de Buenos Aires.
- Kleinberg, Jon y Éva Tardos (2005). *Algorithm Design*. Boston: Pearson Education.